

Claude Code + Design

Claude Code for Designers

A Designer's Guide to AI-Assisted Workflows

TOOL: CLAUDE CODE + DESIGN
VERSION: 0.1.0 · DATE: MAY 2026

Mehran Mozaffarigithub.com/imehr

This book was written with the assistance of AI tools. All tool references and code examples reflect capabilities as of May 2026. Claude Code features evolve rapidly; check official documentation for the latest capabilities.

Contents

Letter Why I Wrote This Book for Designers

A letter about the gap between design thinking and code execution

1 The Design-Code Gap

Why designers need AI coding tools, what Claude Code is, and where it fits in your workflow

2 Getting Started (No Terminal Fear Required)

Install Claude Code, have your first conversation, and understand what you are looking at

3 Your First Design Prototype

Describe a design, watch Claude build it, and learn to iterate

4 The Art of Prompting for Design

How to communicate visual intent to an AI that thinks in code

5 Teaching Claude Your Design Language

CLAUDE.md for designers, design tokens, brand guidelines, and component conventions

6 Design Systems at Scale

Using Claude Code to build, extend, and maintain component libraries and design systems

7 Automating Design Workflows

Scheduled tasks, batch operations, hooks, and CI integration for design teams

8 Connecting Your Design Tools

MCP servers, Figma integration, browser testing, and building your tool integration map

9 From Design to Production

The capstone: take a design concept through to production-ready code

10 The AI Design Tool Landscape

Claude Code vs Cursor, Copilot, v0, Bolt, and the rest

A Claude Code Command Reference for Designers

The commands, shortcuts, and settings you will actually use

B Pricing, Plans, and Limits

What it costs, which plan to choose, and how to manage your usage

C Troubleshooting and FAQ

Fixes for common problems, answers to frequent questions

0 Why I Wrote This Book for Designers

A letter about the gap between design thinking and code execution

0.1 The Wall Between Design and Code

There is a wall between your design vision and the working product, and you have lived on one side of it for your entire career.

Figure 00.1: The design-code gap: where design intent gets lost between tools

You know the feeling. You have a design fully resolved in your head — the spacing, the hierarchy, the way the hover state transitions into the active state. But to make it real, you need someone else to write the code.

So you annotate. You write specs. You sit in handoff meetings explaining what you mean by "generous whitespace." The engineer nods, interprets your vision through their own lens, and builds something close. The spacing drifts. The typography shifts. You file feedback. Days pass.

What was clear in your head is now a negotiation. This is the wall. Every designer I have ever worked with knows it.

The handoff tax

The design industry has built entire product categories to deal with this wall. Zeplin, Avocode, Sympli, InVision Inspect — tools that exist to translate design files into something engineers can build from. The tools keep changing because the underlying problem has not: designers think visually, engineers think in code, and someone has to bridge the gap.

My Take

The most impactful skill a designer can develop right now is not another design tool — it is learning to use AI to execute their own ideas. The gap between design intent and implementation has always been the biggest bottleneck in product development. For the first time, that gap is crossable without an engineering degree.

Why I believe this: I have watched designers go from "I can't code" to "I built a working prototype this morning" in weeks, not months. The bottleneck was never ability. It was access to the execution layer.

Caution: Strong opinion based on observation, not survey data.

0.2 Why I Started Using Claude Code

I started using Claude Code because I was tired of watching my designs lose fidelity between my screen and the browser.

The first time I opened it, I did not know what to expect. I had tried other AI coding tools and found them either too slow, too inaccurate, or too focused on developers to be useful for design work. Claude Code felt different — not because it was smarter, but because it could work with real files in a real project. It could read existing code, edit multiple files at once, and run commands to see the results.

I described a layout — a hero section with a gradient background, a bold heading, and a call-to-action button. Claude Code generated working HTML and CSS in about ten seconds. The spacing was wrong. The gradient went in the wrong direction. But it was *real* — a file I could open in a browser, close enough to iterate on.

I typed a follow-up: "Make the gradient go from dark indigo to soft blue. Increase the heading size. Add more space between the heading and the button." Claude Code updated the code. I refreshed the browser. Two more rounds of feedback and it matched what I had in my head. That loop — describe, review, refine — took less than five minutes.

Before Claude Code, my path from design to working code went through specs, handoff meetings, and days of waiting. With Claude Code, the middle steps collapsed. I could go from intent to working prototype in one session.

That first session changed how I thought about the wall. Claude Code did not demolish it — but it gave me a way across. My design judgment was still the engine. The difference was that I could now drive.

0.3 What This Book Will Give You

This book teaches you to build real, working design output using Claude Code — without becoming a software engineer.

You will not learn to code in the traditional sense. There are no chapters on variables, loops, or data structures. Instead, you will learn to direct an AI collaborator that writes code based on your design intent. You describe what you want. Claude Code builds it. You review, refine, and iterate. Your judgment drives the process. Claude Code handles execution.

Book Section	What You Will Be Able To Do
Chapters 1–2: Foundations	Understand what Claude Code is, where it runs, and get it installed. No terminal experience required.
Chapters 3–4: Core Skills	Build a working prototype from a description and write prompts that produce good design output consistently.
Chapters 5–7: Your Design System in Code	Translate your brand guidelines, design tokens, and component library into code that Claude Code respects every session.
Chapters 8–10: Advanced Workflows	Automate repetitive tasks, connect your design tools through integrations, and manage your work with version control.

What this book is not

This is not a "learn to code" book. You will not become a software engineer. What you will become is a designer who can execute — someone who can take an idea from concept to working prototype without waiting for someone else. If you want a computer science curriculum, this is the wrong book. If you want to bring your design ideas to life faster, keep reading.

My Take

Claude Code's biggest advantage over AI design tools like v0 or Figma AI is that it produces real, editable, deployable code — not a screenshot, not a dead-end prototype. The output IS the implementation. You never hit a wall where your prototype cannot go further.

Why I believe this: I have built prototypes in Figma, v0, and Claude Code. Only the Claude Code prototypes could be handed to an engineering team without being rebuilt from scratch.

Caution: Code quality depends on your prompts and iteration.

0.4 How to Read This Book

Read the first three chapters in order. After that, follow the path that matches your work.

Figure 00.2: Three reading paths through the book, depending on your goals

Chapters 1 through 3 build on each other. Chapter 1 explains what Claude Code is. Chapter 2 gets it installed. Chapter 3 walks you through building your first prototype. These three are the on-ramp — do not skip them, even if you have already tried Claude Code on your own.

From Chapter 4 onward, you have choices. The chapters are organized by skill, and different designers will take different paths.

If You Are	Start With	Then Go To
New to any coding tool	Chapters 1–3 in order	Chapter 4, then Chapter 5
Comfortable with terminal or CLI	Chapter 1 (skim), Chapter 3	Chapter 5, then Chapter 8
A design system or brand designer	Chapter 5	Chapter 6, Chapter 7
An interaction or motion designer	Chapter 3	Chapter 7
Looking to automate your workflow	Chapters 1–3	Chapter 8, Chapter 9

You do not need to know any programming language to follow along. Every technical concept is introduced through a design-world analogy first. When I mention a code term like `flexbox` or `css grid`, I explain it in Figma terms before using the technical name.

You will get the most from this book if you follow along in Claude Code as you read. Each chapter from Chapter 3 onward includes hands-on exercises for a single sitting. The only prerequisite is a computer (Mac or Windows) and curiosity. Claude Code requires a paid subscription — I will walk you through the options in Chapter 2 before you spend anything.

On staying current

Claude Code is evolving quickly. The core concepts in this book — how to describe design intent, iterate on AI output, encode your design system — will hold up regardless of how the tool changes. Specific features may shift. When that happens, I will post updates at the book's companion site.

Chapter 1 examines the design-code gap from both sides: why it exists, what it costs, and why AI tools have changed the equation for the first time.

01 The Design-Code Gap

Why designers need AI coding tools, what Claude Code is, and where it fits in your workflow

Claude Code is a collaborative AI agent that runs across your tools and can turn design intent into working code, but it works best when you understand its strengths and limits.

1.1 Why Designers Should Care About AI Coding Tools

The gap between what you design and what ships has always existed. AI coding tools are the first thing that lets you cross it on your own terms.

Figure 01.1: Traditional design workflow vs AI-assisted workflow: fewer handoffs, faster iteration

Here is a scenario you might recognize. You have a clear vision for a component, a page, an interaction. You have built it in Figma down to the pixel. You hand it off to engineering. Three days later, what ships looks close but not right. The spacing is off. The hover state is different from what you specified. The responsive behavior collapses in ways you did not anticipate.

This is the design-code gap. It is not anyone's fault. It is a structural problem. Designers think in visual systems. Engineers think in code systems. The handoff between them is lossy, like compressing a high-resolution image into a low-resolution format. Information gets lost.

AI coding tools change this equation. They do not eliminate the gap, but they give you a way to cross it yourself, without waiting for someone else to translate your intent into code. You describe what you want in natural language, paste a screenshot, or point to a reference file. The tool generates working code. You iterate on the output. You stay in control of the design decisions while the tool handles the code execution.

Warning

This is not about replacing engineers. This is about expanding what you can do as a designer. The best results come when designers and engineers collaborate, with AI tools accelerating the parts that slow both sides down.

The scope of design work has been expanding for years. Product designers now maintain design systems in code. UX researchers build interactive prototypes instead of static wireframes. Brand designers create email templates and landing pages that live in code repositories. The line between "designer" and "front-end developer" has been blurring, and AI coding tools are accelerants for that shift.

Figma, the company that makes the design tool most of us use daily, is a customer of Claude Code. Their logo appears on Anthropic's product page. That is not a coincidence. Design-focused companies see the value in AI-assisted coding, and individual designers should too.

In my experience, the designers who benefit most from AI coding tools are not the ones trying to become developers. They are the ones who want to move faster, test ideas sooner, and reduce the friction between thinking and building. Claude Code is not the only tool in this space, but it is the one I have found most effective for design work, and the one this book focuses on.

1.2 What Claude Code Actually Is

Claude Code is an agentic coding tool. Not a chatbot. Not a design tool. A collaborator that reads your files, runs commands, and builds things.

Figure 01.2: A first Claude Code conversation: asking what it can do for designers

Anthropic defines Claude Code as "an agentic coding tool that reads your codebase, edits files, runs commands, and integrates with your development tools." That is accurate but dense. Let me translate.

Think of Claude Code like a junior developer who sits in your tools and follows your instructions. You tell this developer what you want built. They read the relevant files, figure out how things work, write the code, run tests to verify it, and show you the result. You review, give feedback, and they iterate. That loop, where the tool reads your project, plans an approach, executes changes, and verifies the result, is the agentic loop. It is what separates Claude Code from a chatbot that only answers questions.



Here is what that looks like in practice. You open Claude Code, point it at a project folder, and type a prompt. For example:

```
Starting a session bash
claude "Describe what this project does"
```

Claude Code reads the files in your project, builds an understanding of the structure, and responds with a summary. If you ask it to make changes, it reads the relevant files, writes the edits, and shows you a diff, a side-by-side comparison of what changed, like reviewing track changes in a document.

You control how autonomous Claude Code is. Five permission modes let you dial this in:

Permission Modes in Claude Code

Mode	What Happens	When to Use
Default	Asks before each action	Learning the tool, first time with a project
Auto accept edits	Auto-approves file edits, asks for commands	Comfortable with changes, want to move faster
Plan mode	Read-only exploration, no changes made	Auditing a codebase, understanding structure
Auto mode	AI safety classifier decides	Trusted projects, repetitive tasks
Bypass permissions	No prompts, sandboxed only	Automated pipelines, CI/CD

Start in Default mode. You will see every action before it happens, which builds trust and understanding. As you get comfortable, you can relax the permissions to move faster.

Key Distinction

Claude Code is a collaborator, not a magic button. It produces better results when you give it clear direction, review its output critically, and iterate. The designer who treats it like a conversation partner gets more from it than the one who expects a finished product from a single prompt.

1.3 The Claude Code Ecosystem Map

Claude Code meets you wherever you work. Six primary surfaces, all connected to the same engine.

Figure 01.3: The Claude Code ecosystem: five surfaces, each with different designer-friendliness levels

One of the most common questions I hear from designers is: "Where do I even open this thing?" The answer depends on what feels comfortable to you, because Claude Code runs in more than one place.

Think of it like Figma. You can open Figma in a browser, in the desktop app, or on your phone. The files are the same. The experience differs slightly depending on where you are. Claude Code works the same way. Your projects, settings, and preferences follow you across surfaces.

Claude Code Surfaces for Designers

Surface	Best For	Designer Friendliness	Requires
Desktop app	Visual diff review, app preview, scheduled tasks	High	macOS or Windows
VS Code extension	Inline editing, live preview, file-level context	Medium-High	VS Code (free)
Web (claude.ai)	Quick tasks, no-install access, mobile handoff	High	Browser + subscription
Terminal CLI	Full control, scripting, automation	Low	Terminal comfort
JetBrains plugin	IntelliJ, WebStorm users	Medium	JetBrains IDE
iOS app	Remote monitoring, quick check-ins	Medium	iPhone + subscription

In addition to the six primary surfaces, Claude Code integrates with Slack (route tasks from team chat to code changes via @Claude mentions) and has a Chrome extension for debugging live web applications and comparing visual output. These are supplementary. The six surfaces above are where you will spend most of your time.

My Take

My take: for designers, the Desktop app or VS Code extension is the best starting point, not the terminal. The terminal is faster for experienced users, but the Desktop app gives you visual diff review without learning CLI conventions, and VS Code gives you inline editing in an environment that at least has a graphical interface. Start where you are comfortable. You can always try the terminal later. Sessions are portable across surfaces. You can start a task on the web, pull it into the desktop app with `/desktop`, or check on progress from your phone using Remote Control. This matters for designers who move between their desk, meetings, and their phone throughout the day.

One practical note: Claude Code requires a paid subscription or API account. The entry point is Claude Pro at \$17 per month (annual billing) or \$20 per month (monthly). Higher tiers, Max 5x at \$100 per month and Max 20x at \$200 per month, give you more usage capacity. The free Claude tier does not include Claude Code access.

Installing Claude Code (macOS)

[bash](#)

```
curl -fsSL https://claude.ai/install.sh | bash
```

That command installs the CLI. The Desktop app and VS Code extension have their own installers and are covered in detail in Chapter 2.

1.4 How Claude Code Fits Your Design Workflow

Claude Code is not a replacement for your design process. It is an accelerant at specific stages where code execution is the bottleneck.

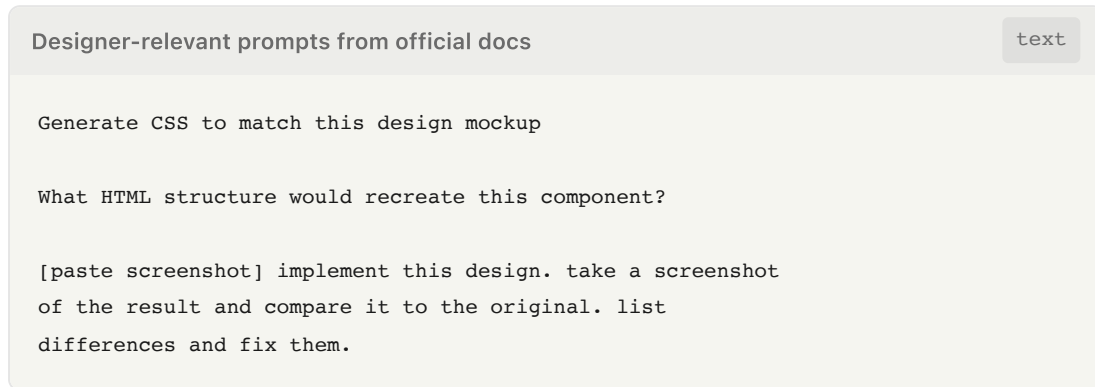
Figure 01.4: Where Claude Code fits in the standard design process: strongest at prototyping and testing

Every designer has a process. It might be formal or loose, but it usually follows a pattern: research, ideation, prototyping, testing, and handoff or implementation. Claude Code fits into specific stages of that process. It does not replace the thinking. It accelerates the execution.

Design Process Stage to Claude Code Feature

Design Stage	Claude Code Feature	Example
Research	Codebase exploration, file search	"Summarize how the current design system implements buttons"
Ideation	Quick prototype generation from descriptions	"Build a card component with a hero image, title, and CTA"
Prototyping	Interactive prototype from screenshots or specs	"Recreate this Figma frame as a responsive HTML page"
Testing	Visual verification, cross-browser screenshots	"Take a screenshot and compare it to this reference image"
Handoff	Production-ready code generation	"Refactor this prototype into a reusable React component"

The most powerful integration point is the design-to-code pipeline. Claude Code can accept image inputs: screenshots, mockups, wireframes, reference designs. You paste a screenshot of a Figma frame, and Claude generates the HTML, CSS, and JavaScript to implement it. Here are some prompts that work well for design work, taken directly from the official documentation:



The last prompt is worth pausing on. Claude Code can take a screenshot of its own output, compare it to your reference, identify differences, and fix them. This is the visual verification loop, and it is the closest thing to having a design review conversation with a tool. You show it what you want. It builds something. It compares its work to your reference. It improves. You are the quality gate throughout.

Tip

For the strongest results, paste a screenshot alongside a text description. The screenshot gives Claude the visual target. The text gives it the constraints and preferences that a screenshot alone cannot convey: font family names, specific color values, responsive breakpoints, accessibility requirements.

Claude Code also connects to external tools through MCP (Model Context Protocol). Think of MCP like plugins for your design tool. Each one adds a new connection. You can connect Claude Code to Figma for design data, to Notion for documentation, to Slack for team communication, and to databases or APIs for live data. The connection is made with a single command:

Connecting to Figma via MCP

bash

```
claude mcp add --transport http figma \  
https://api.figma.com/mcp/ \  
--header "Authorization: Bearer YOUR_FIGMA_TOKEN"
```

Once connected, Claude Code can read Figma file data directly. This is not the same as editing Figma files, which Claude Code cannot do. But it means Claude can pull design tokens, component specs, and layout information from your Figma project and use them to generate consistent code.

My Take

My take: Claude Code's biggest advantage over dedicated AI design tools like v0 or Figma's built-in AI is that it produces real, editable, deployable code. Designs that live in Figma still need to be implemented by an engineer. Claude Code's output is the implementation. You can ship it, commit it to version control, and iterate on it directly. That is a different category of output from a generated screenshot or mockup.

1.5 What Claude Code Can and Cannot Do

Honest assessment. Claude Code can do a lot, but it has real boundaries. Knowing them saves you frustration.

Figure 01.5: What Claude Code can and cannot do for design work: an honest assessment

Every tool has a gap between what it promises and what it delivers. The gap narrows with practice, but it never fully closes. Here is what I have found Claude Code does well, where it falls short, and what to watch for.

Capabilities for Design Work

Task	Can Claude Do It?	Quality Level	Caveats
Generate HTML/CSS from a description	Yes	Good to very good	First output is a starting point; 2-4 iterations improve it
Generate code from a screenshot	Yes	Good	Pair screenshot with text for best results
Build interactive prototypes	Yes	Good	Simple interactions are reliable; complex animations need iteration
Verify visual output	Yes	Good	Desktop app and Chrome extension enable screenshot comparison
Maintain design system consistency	Yes, with CLAUDE.md	Very good	Requires a CLAUDE.md file encoding your design rules
Edit Figma files directly	No	N/A	Can read Figma data via MCP but cannot modify Figma documents
Run design tools (Sketch, Adobe XD)	No	N/A	Not a design application; works with code files
Replace design thinking	No	N/A	Your judgment on hierarchy, usability, and aesthetics is irreplaceable

Three limitations deserve more detail.

Context windows fill up. Claude Code has a maximum amount of information it can hold in a single session. Long sessions degrade in quality because the tool loses track of earlier context. The fix is to start fresh between unrelated tasks and keep sessions focused. In Chapter 5, you will learn how to use CLAUDE.md to carry important context across sessions without re-explaining everything.

Output needs your review. Claude Code generates syntactically correct code most of the time. But correct code is not the same as good design. It might choose spacing values that are technically fine but do not match your design system. It

might implement a layout that works but feels wrong aesthetically. You are the quality gate. Every time.

It requires a codebase or project folder. Claude Code works best when pointed at a directory with actual files. It can work in any folder, including a collection of markdown files or a notes vault, but it is not a standalone application. You give it a project, it works within that project.

Warning

Claude Code is not Figma. It does not have a canvas, a layers panel, or a visual editor. It works with code. But the code it produces creates visual output that you can preview in a browser, compare to your designs, and iterate on until it matches your intent. Your design judgment is essential. The tool accelerates execution; it does not replace taste.

Here is a practical example of how to use Claude Code for a common design task. Say you have a folder with a single HTML file and you want to generate a design tokens file from a screenshot of your brand guidelines:

Design token generation prompt

text

[paste screenshot of brand guidelines]

Extract the design tokens from this brand guide and create a CSS file with custom properties for:

- All brand colors (primary, secondary, neutral scale)
- Typography scale (heading sizes, body size, line heights)
- Spacing scale
- Border radius values
- Shadow values

Claude Code reads the screenshot, identifies the colors, sizes, and spacing in your brand guidelines, and generates a CSS file with custom properties. You review, adjust values that do not match, and save. What used to take a manual pass through a brand PDF now takes a single prompt and a review cycle.

My Take

My take: the biggest mindset shift for designers using Claude Code is treating AI output like a first draft from a junior developer. You would not expect a junior developer to nail your design on the first try. You would give them feedback, and they would improve. Claude Code works the same way. Your value is in the design direction and the quality review, not in the initial code generation.

You now have a mental model for what Claude Code is, where it runs, and what it can do for your design work. The next step is getting it running on your machine. Chapter 2 walks through installation with zero terminal experience required.

2. Getting Started (No Terminal Fear Required)

Install Claude Code, have your first conversation, and understand what you are looking at

You can get Claude Code running in under ten minutes without ever opening a terminal, and your first interaction will show you exactly how it works.

In Chapter 1, you saw what Claude Code is and where it fits in a design workflow. Now it is time to get it running on your machine. This chapter walks you through choosing the right interface, installing, signing in, having your first conversation, and setting up a project folder.

If the word "terminal" makes you tense, good news: you do not need one. The Desktop app gives you a visual interface with buttons, menus, and a file browser. I recommend it as your starting point.

Before you start

Claude Code requires a paid Anthropic subscription (Pro, Max, Team, or Enterprise). The free Claude.ai plan does not include Claude Code access. If you are on the free plan, upgrade at claude.ai/settings before proceeding. You also need to be in a country where Anthropic services are available. Check the list at anthropic.com/supported-countries.

2.1 Choosing Your Entry Point

Claude Code runs on three primary surfaces, and the right one depends on how comfortable you are with developer tools.

Figure 02.1: Choosing your entry point: Desktop app is recommended for most designers

All three surfaces connect to the same Claude Code engine. Your CLAUDE.md files, settings, and connected tools work across all of them. You are not locked into one choice. But your first experience matters, and I want you to start where the friction is lowest.

Here is how the three options compare for designers:

Surface	Comfort Level	Setup Time	Best Feature for Designers
Desktop app	No coding experience needed	5 minutes	Visual diff review, drag-and-drop layout, app preview
VS Code extension	Some familiarity with code editors	5 minutes	Inline editing, @-mentions for files, live preview
Terminal CLI	Comfortable with command line	5-10 minutes	Fastest workflow, full control, scripting

The Desktop app has a visual interface you navigate with clicks and menus, the same way you use Figma or any other design tool. It shows you a side-by-side comparison before any file is changed. It has a built-in file browser, a preview pane where you can see your running design, and a layout you can rearrange by dragging panes around.

VS Code is a strong second choice if you already use it for editing CSS or inspecting code. The extension integrates directly into the editor, so you can highlight code and ask Claude about it.

The terminal is the most powerful surface, but it is also the most intimidating for designers who have never used one. It is there when you want it, and later chapters cover it for specific tasks. For now, it is not your starting point.

My Take

If you are a designer who has never used a terminal, start with the Desktop app. Do not force yourself through the CLI. The Desktop app gives you visual diff review, a graphical file browser, and permission buttons you can click instead of typing "y" into a black box. You can always switch to the terminal later when you are curious, not because you have to.

Why: I have watched dozens of designers bounce off Claude Code because they started with the terminal and hit a wall of unfamiliar conventions. The ones who started with the Desktop app kept going.

Caution: Power users and developers on your team will prefer the terminal for speed. That is fine. Different tools for different comfort levels.

2.2 Installation Walkthrough

Step-by-step installation for the Desktop app, with fallback instructions for VS Code and the terminal.

I am going to walk you through the Desktop app installation first because it is the most straightforward path for designers. After that, I will cover VS Code and the terminal as alternatives.

Path A: Desktop App (Recommended)

The Desktop app is a standalone application you download and install, the same way you installed Figma or Sketch. No terminal, no package managers, no prerequisites beyond a paid Anthropic account.

1 Download the installer.

Open your browser and go to the download page for your operating system:

- **macOS** (Intel and Apple Silicon):
`claude.ai/download`
- **Windows** (x64):
`claude.ai/download`

The page detects your OS automatically and gives you the correct installer.

2 Install the application.

- **macOS:**
Open the downloaded DMG file. A window appears with the Claude icon and your Applications folder. Drag the Claude icon into the Applications folder.
- **Windows:**
Double-click the downloaded installer executable. Follow the setup wizard prompts.

3 Launch the app and sign in.

Open Claude from your Applications folder (macOS) or Start menu (Windows). Sign in with your Anthropic account — the same one you use at `claude.ai`.

4 Open the Code tab.

At the top center of the app window, you see three tabs: **Chat**, **Cowork**, and **Code**. Click **Code**. This is where all your design work happens. If you see a prompt to upgrade your subscription, your current plan does not include Claude Code. You need Pro, Max, Team, or Enterprise.

5

4

Sign in. A sign-in screen appears in the Claude panel. Click **Sign in** and complete the authorization in your browser. After you authorize, return to VS Code.

If the extension does not appear after installation, restart VS Code or run **Developer: Reload Window** from the Command Palette.

Path C: Terminal CLI

The terminal is the fastest path once you are comfortable with it. For now, I am including it for completeness. You can skip it and come back later.

mac
and
Linu

`command not found`, the install directory is not in your PATH. Run this fix:

```
echo 'export PATH=  
source ~/.zshrc  
claude --version
```

For a thorough check, run the diagnostic command:

```
claude
```

Regardless of which surface you choose, the next steps work the same way. You type what you want, Claude responds, and you decide whether to accept the change.

2.3 Your First Conversation with Claude

Using Claude Code is a conversation, not a command line. You describe what you want, and Claude responds with a plan and actions.

Figure 02.3: Your first conversation: Claude explains what it can do for designers

Think of it like giving a brief to a junior developer. You would not hand them a single word and expect great results. You would describe the intent, the constraints, and the outcome you want. The same principle applies here.

Let me walk you through your first interaction. Open Claude Code in whichever surface you installed.

Desktop App: Your First Prompt

1

Make sure you are in the Code tab with a folder selected. If you created `my-design-project` earlier, select that folder.

Now let me show you what happens when you ask Claude to do something:

```
Create an index
```

When you send this prompt, several things happen in sequence:

1

Claude thinks. You see a brief moment where Claude considers your request. In the Desktop app, you might see a "thinking" indicator. In the terminal, you see a spinner.

2

Claude proposes a change. A diff view appears showing the changes Claude wants to create or modify. In the Desktop app, the change shows up as a new file with green highlighted text. In the VS Code, you see a side-by-side diff panel.

3

Type your prompt in the input box at the bottom of the Claude panel:

```
Create an index.html file with a centered
```

4

Press Enter.
You see a different
Reject.

```
> Create an index.htm
```

Claude shows the proposed change and asks for confirmation. Press **y** to accept or **n** to reject.

Congratulations. You have completed your first Claude Code interaction. You described what you wanted, reviewed what Claude proposed, and decided whether to apply it. That's the fundamental loop — describe, review, accept or reject, iterate. Every session follows this pattern.

2.4

Understanding What You Are Looking At

The Claude Code interface has several elements, and each one carries specific information about your conversation and your project.

Figure 02.4: Annotated VS Code interface showing every Claude Code element a designer needs to know

If you are used to Figma, you are accustomed to a canvas with layers, properties, and tool panels. Claude Code has its own layout, and once you understand what each element shows, the interface feels familiar rather than overwhelming.

Desktop App Interface

What Happens	When to Use	
Ask permissions	Claude asks before every file edit and command	Your first sessions. You see every change before it happens.
Auto accept edits	File edits are applied automatically; commands still ask	When you trust the output and want faster iteration
Plan mode	Claude reads files and proposes a plan without editing	Complex tasks where you want to review strategy first
Auto	Background safety checks verify alignment	Experienced users with Max/Team/Enterprise plans

Start with **Ask permissions**. You will see every proposed change before it touches your files. Switch to other modes when you feel confident about the output quality.

VS
Ext
Int

Keyboard Shortcuts Worth Knowing	You do not need to memorize these, but they speed things up once you are comfortable:	Desktop app (Mac):			VS Code extension:
			Cmd+N	New	
			Cmd+W	Close	
			Esc	Stop	
			Cmd+Shift+D	Toggle	
			Cmd+Shift+P	Toggle	
			Cmd+ /	Search	

2.5 Setting Up for Design Work

Create a project folder that becomes your workspace for the prototype exercise in Chapter 3.

Every Claude Code session starts in a folder. That folder is your workspace — the place where Claude reads files, creates files, and makes changes. Think of it like a Figma project file: everything related to one piece of work lives in one place.

Creating Your Project Folder

Desktop app:

1 Create a new folder on your Desktop called `my-design-project`. You can do this in Finder (Mac) or Explorer (Windows) — right-click, select **New Folder**, and name it.

2 In the Claude Desktop app, click **Select folder** in the Code tab. Navigate to your Desktop and select `my-design-project`.

VS Code:

For now, let me show you how to create a basic one. In your Claude Code session, type:

```
Create a CLAUDE.md
- Use semantic HTML
- Use CSS custom properties
- Write clean, well-organized code
- When building UI components
```

Claude creates the file and shows you the diff. Accept the change. Now, every time you start a new session in this folder, Claude reads those instructions and follows them.

Chapter 5 goes deeper into design systems — design tokens, typography scales, color conventions. For now, this gives you a foundation.

Verifying Your Setup

Before moving on, confirm everything is working. In your Claude Code session, type:

Read the C

Claude should respond by reading the file and summarizing the instructions back to you. If it does, everything is connected and working. You have a workspace, Claude understands your preferences, and you are ready to build.

My Take

Setting up a project folder and a CLAUDE.md file before you build anything is the single best investment of your time in this chapter. Every session after this benefits from that file. Designers who skip this step end up re-explaining their preferences in every conversation. Those who take five minutes to create it get consistent output from day one.

Why: Claude Code has no memory between sessions. The CLAUDE.md file is its memory. Without it, every session starts from scratch.

Caution: Do not over-engineer your CLAUDE.md at this stage. A few clear instructions are better than a comprehensive document you never update.

Your First Design Prototype

Describe a design, watch Claude build it, and learn to iterate

You have Claude Code installed. You have a project folder. Now you are going to build something real — a hero section for a product landing page, starting from a description in your head and ending with a working prototype you can see in your browser. This chapter walks through every step.



That is the design loop. Everything in this book builds on it. By the end of this chapter, you will have run through it three times and built something you can open in a browser. Not a screenshot. Not a mockup. A real, working HTML file.

3.1 What We Are Building

We are building a hero section — the top banner of a landing page. If you have designed one in Figma, you know exactly what this is.

Figure 03.1: The design brief: a hero section with heading, subheading, and CTA button

A hero section is the right first project for three reasons. You already know what a good one looks like, so you can judge the output. It involves layout, typography, color, and spacing — the fundamentals of visual design. And it can live in a single HTML file with no dependencies, no build tools, no npm.

Here is the design brief:

Element	Specification
Background	Full-viewport gradient, dark navy to slate blue
Heading	Large white text: "Build faster. Ship smarter."
Subheading	Lighter gray, smaller: "The AI-powered tool that understands your codebase"
CTA button	Blue (#3b82f6), white text, rounded corners, hover effect
Layout	Vertically and horizontally centered
Font	System font stack (no external fonts needed)

Nothing exotic. A hero section you have designed a hundred times. The difference is that this time, you are going to describe it to Claude Code instead of building it yourself.

Why a single HTML file?

This chapter deliberately uses plain HTML with inline CSS. No frameworks, no build tools, no React. The goal is to learn the design loop, not to learn a framework. Single-file HTML is the fastest path from idea to browser. Frameworks come later in the book, once you are comfortable with the loop.

3.2 Describing Your Design Intent

The prompt is where your design skill meets Claude's code ability. How you describe what you want directly determines what you get back.

Figure 03.2: Vague prompt vs specific prompt: the output quality difference is dramatic

Think of the prompt like a design brief you would hand to a developer. A vague brief produces generic work. A specific brief produces work that matches your intent. The same principle applies here.

